

## LABORATORIO 02

# Cómo está hecha la libreta

Guía para leer mientras trabajas el cuaderno. Cada celda trae la pregunta que responde, la idea en el almacén, la sentencia explicada parte por parte, lo que sale en pantalla y cómo leerlo. Las salidas son las de la solución ejecutada del repositorio.

# Introducción

## 1 · El tema

Una tabla Iceberg recuerda cada versión de sí misma, y no tiene ninguna columna con versiones. En este laboratorio abres la tabla por dentro para ver dónde está guardada esa memoria. Al final compruebas lo más útil de esa construcción, que dos motores distintos leen y escriben la misma tabla sin coordinarse.

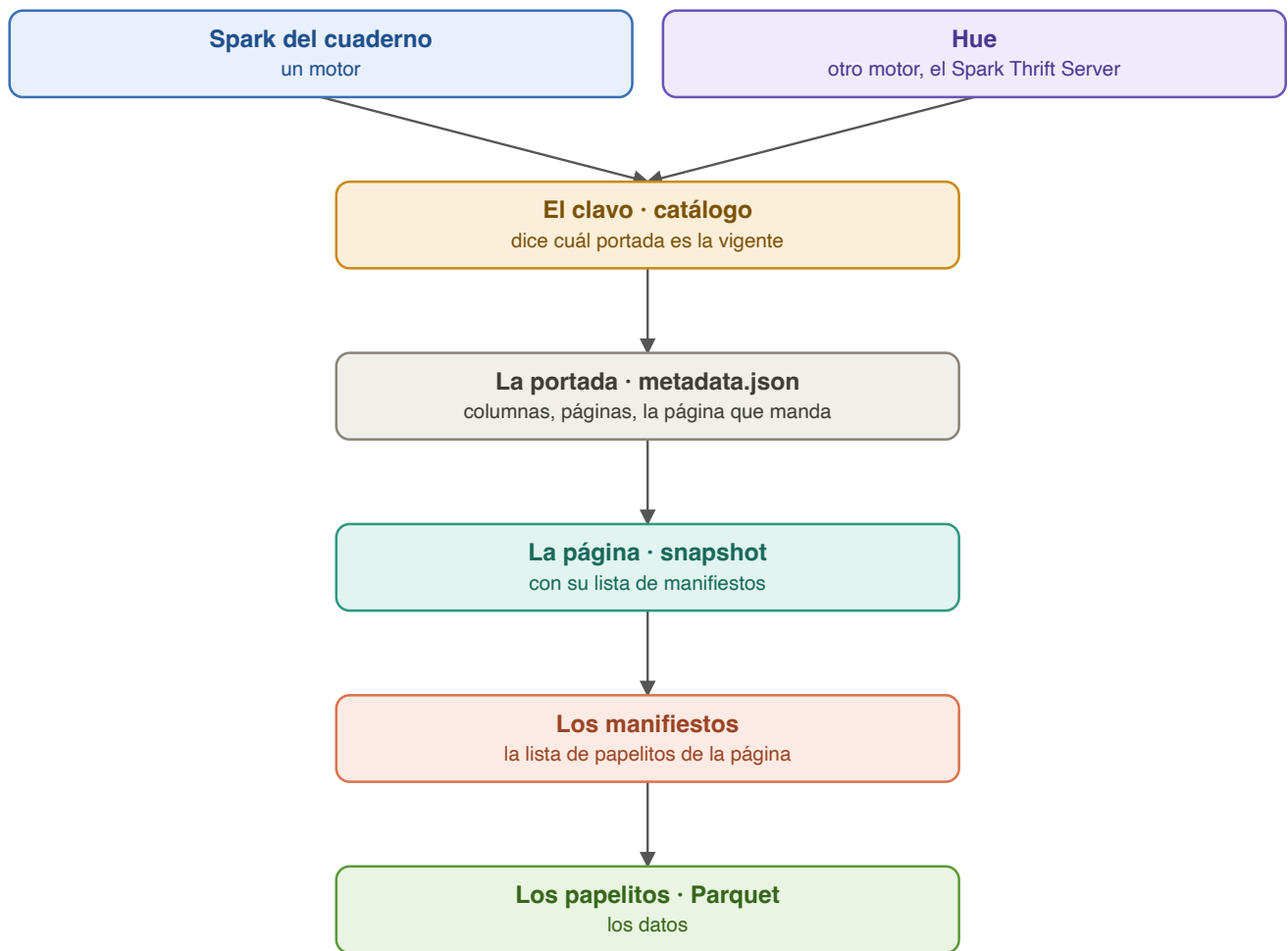
## 2 · El problema, en el almacén

En un almacén grande trabajan varias personas, y cada una lleva su propia copia de la libreta. Cuando uno anota algo, los demás no lo ven hasta que alguien pasa las copias en limpio. Al final del día las copias no cuadran, y nadie sabe cuál es la verdadera.

## 3 · El problema en términos técnicos

Una plataforma de datos tiene varios motores de consulta, por ejemplo Hive, Impala y Spark. Si cada motor guarda su propia idea de qué es una tabla, qué columnas tiene y qué archivos la forman, una tabla que uno lee bien el otro la lee distinto. Para que dos motores vean lo mismo, la definición de la tabla no puede vivir dentro de ninguno de ellos.

## 4 · El diagrama



**Spark del cuaderno** (azul) y **Hue** (morado). Son dos motores distintos. Ninguno guarda la tabla adentro. Los dos van a buscarla al mismo lugar.

**El clavo · catálogo** (amarillo). Es el Hive Metastore. Lo único que sabe de la tabla es dónde está su portada vigente.

**La portada · metadata.json** (gris). Es un archivo con la definición de la tabla, sus columnas, la lista de sus páginas y cuál de ellas manda ahora.

**La página · snapshot** (verde agua). Cada escritura deja una página. Cada página apunta a una lista de manifiestos.

**Los manifiestos** (coral). Cada manifiesto es una lista de papelitos, los archivos de datos que pertenecen a la página, con cuántas filas tiene cada uno.

**Los papelitos · Parquet** (verde). Son los archivos con los datos, en formato Parquet.

## 5 · La solución

En el almacén hay una sola libreta, colgada de un clavo en la pared. Cualquiera que mire el clavo lee la misma libreta. La libreta tiene una portada que dice cuál es la página vigente, cada página tiene una lista de las anotaciones que le pertenecen, y las anotaciones están en papelitos.

En Iceberg la tabla es un conjunto de archivos guardados en el almacenamiento, más un catálogo que apunta a la portada vigente. La definición, las páginas, los manifiestos y los datos están en archivos, no dentro de un motor. Por eso dos motores que miran el mismo catálogo ven exactamente la misma tabla.

## 6 · Los pasos

- **Paso 0.** Creas la tabla y la cargas en dos veces, para tener dos páginas.
- **Paso 1.** Le pides a la tabla que se describa y miras sus propiedades.
- **Paso 2.** Miras las páginas, el historial y los manifiestos.
- **Paso 3.** Miras los papelitos y cuánto pesan.
- **Paso 4.** Escribes una fila desde Hue y la ves aparecer en el cuaderno.

Si quieres repetir el laboratorio desde el principio, corre este comando desde la carpeta del repositorio. Para el paso 4, el ambiente tiene que estar arriba con Hue, que se levanta con `bin/ambiente.sh arriba --sql`.

```
bash
bin/reiniciar-lab.sh 02
```

## Paso 0 · Preparar la tabla

### Celda 0.1 · Ponerte en tu espacio

**La pregunta.** ¿En qué espacio vas a trabajar?

**Por qué ahora.** La tabla tiene que quedar en tu espacio, `mi_espacio`.

**En el almacén.** Es ponerte frente a tu estante.

**La sentencia, parte por parte.**

- `%%sql` es la primera línea de toda celda SQL del cuaderno. Le dice al cuaderno que lo que sigue es SQL.
- `-- Celda 0.1` es un comentario. El motor ignora lo que va después de dos guiones.
- `USE mi_espacio` cambia el espacio activo. Las tablas sin espacio adelante se buscan ahí.

```
sql
%%sql
-- Celda 0.1
USE mi_espacio
```

**Lo que sale en pantalla.**

```
text
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/09/25 18:06:03 WARN NativeCodeLoader: Unable to load native-hadoop library for your
platform... using builtin-java classes where applicable
Listo. La sentencia se ejecutó.
```

**Cómo se lee.** Quedaste en `mi_espacio`. La última línea es el aviso del cuaderno de que la sentencia terminó bien.

Las tres primeras líneas no son errores. Salen solo en la primera celda, cuando arranca Spark.

- `Setting default log level to "WARN"` dice que Spark va a mostrar solo los avisos y los errores.
- `To adjust logging level use sc.setLogLevel(newLevel)` explica cómo cambiar eso. `sc` es el contexto de Spark y `setLogLevel` la función que cambia el nivel. SparkR es la versión de Spark para el lenguaje R, que aquí no se usa.
- `WARN NativeCodeLoader: Unable to load native-hadoop library` dice que Spark no encontró una biblioteca de Hadoop escrita para este sistema operativo y usa la versión en Java. Funciona igual. `26/09/25 18:06:03` es la fecha y la hora del aviso, en UTC.

## Celda 0.2 · Partir de cero

**La pregunta.** ¿Cómo te aseguras de partir con la tabla vacía?

**Por qué ahora.** Si ya corriste el laboratorio, la tabla existe y el `CREATE TABLE` fallaría.

**En el almacén.** Es sacar del estante la libreta vieja con ese nombre, si la hay.

**La sentencia, parte por parte.**

- `DROP TABLE` borra una tabla.
- `IF EXISTS` hace que no se queje si la tabla no existe.
- `contribuyentes_lab02` es la tabla del laboratorio.

```
sql
%%sql
-- Celda 0.2
DROP TABLE IF EXISTS contribuyentes_lab02
```

**Lo que sale en pantalla.**

```
text
Listo. La sentencia se ejecutó.
```

**Cómo se lee.** Terminó bien. Si la tabla no existía, no hizo nada.

## Celda 0.3 · Crear la tabla

**La pregunta.** ¿Cómo es la tabla de hoy?

**Por qué ahora.** Hace falta una tabla Iceberg para mirarla por dentro.

**En el almacén.** Es abrir una libreta nueva con tres columnas.

**La sentencia, parte por parte.**

- `CREATE TABLE contribuyentes_lab02` crea la tabla en tu espacio.
- `rut STRING`, `razon_social STRING` y `segmento STRING` son tres columnas de texto.
- `USING iceberg` hace que sea una tabla Iceberg.

```
sql
%%sql
-- Celda 0.3
CREATE TABLE contribuyentes_lab02 (
  rut          STRING,
  razon_social STRING,
  segmento     STRING
) USING iceberg
```

**Lo que sale en pantalla.**

```
text
Listo. La sentencia se ejecutó.
```

**Cómo se lee.** La tabla quedó creada y vacía.

## Celda 0.4 · El primer lote

**La pregunta.** ¿Qué datos entran primero?

**Por qué ahora.** Vas a cargar la tabla en dos veces, para que queden dos páginas que mirar.

**En el almacén.** Es anotar tres renglones. Queda la página 1.

**La sentencia, parte por parte.**

- `INSERT INTO contribuyentes_lab02` agrega filas.
- `VALUES` introduce las filas escritas a mano, cada una entre paréntesis y separadas por comas.
- Cada fila trae RUT, razón social y segmento.

```
sql
%%sql
-- Celda 0.4
INSERT INTO contribuyentes_lab02 VALUES
  ('77746521-K', 'Pehuen Logistica EIRL', 'MICRO'),
  ('77884562-8', 'Araucaria Ferreteria SpA', 'PEQUENA'),
  ('78800840-6', 'Copihue Maquinarias EIRL', 'MEDIANA')
```

**Lo que sale en pantalla.**

```
text
Listo. La sentencia se ejecutó.
```

**Cómo se lee.** Las tres filas quedaron escritas.

## Celda 0.5 · El segundo lote

**La pregunta.** ¿Qué pasa cuando llega otra carga?

**Por qué ahora.** Una segunda carga deja una segunda página, como pasa con las incorporaciones al padrón que llegan después.

**En el almacén.** Es anotar un renglón más. Queda la página 2.

**La sentencia, parte por parte.**

- `INSERT INTO contribuyentes_lab02 VALUES` agrega una fila, la de Huemul Alimentos.

```
sql
%%sql
-- Celda 0.5
INSERT INTO contribuyentes_lab02 VALUES
  ('76171162-8', 'Huemul Alimentos EIRL', 'GRANDE')
```

**Lo que sale en pantalla.**

```
text
Listo. La sentencia se ejecutó.
```

**Cómo se lee.** La fila quedó escrita.

## Celda 0.6 · Mirar lo cargado

**La pregunta.** ¿Qué quedó en la tabla?

**Por qué ahora.** Para partir sabiendo qué hay.

**En el almacén.** Es leer la libreta como está.

**La sentencia, parte por parte.**

- `SELECT *` pide todas las columnas.
- `FROM contribuyentes_lab02` es la tabla.
- `ORDER BY rut` ordena por RUT, para que salgan siempre en el mismo orden.

```
sql
%%sql
-- Celda 0.6
SELECT * FROM contribuyentes_lab02 ORDER BY rut
```

**Lo que sale en pantalla.**

```
text
rut      razon_social      segmento
76171162-8  Huemul Alimentos EIRL  GRANDE
77746521-K  Pehuen Logistica EIRL  MICRO
77884562-8  Araucaria Ferreteria SpA  PEQUENA
78800840-6  Copihue Maquinarias EIRL  MEDIANA
4 filas.
```

**Cómo se lee.** Cuatro contribuyentes, cargados en dos veces.

# Paso 1 · La tabla se describe a sí misma

## Celda 1.1 · Pedirle a la tabla que se describa

**La pregunta.** ¿Qué es esta tabla como objeto, más allá de sus datos?

**Por qué ahora.** Antes de mirar páginas y archivos, conviene ver qué dice la tabla de sí misma.

**En el almacén.** Es leer la portada de la libreta, donde dice qué columnas tiene, dónde está guardada y de qué tipo es.

**La sentencia, parte por parte.**

- `DESCRIBE TABLE` pide la descripción de la tabla.
- `EXTENDED` pide además la información detallada, no solo las columnas.
- `contribuyentes_lab02` es la tabla.

```
sql
%%sql
-- Celda 1.1
DESCRIBE TABLE EXTENDED contribuyentes_lab02
```

**Lo que sale en pantalla.**

```
text
col_name          data_type
comment
rut                string
razon_social      string
segmento          string

# Partitioning
Not partitioned

# Metadata Columns
_spec_id          int
_partition        struct<>
_file             string
_pos              bigint
_deleted          boolean

# Detailed Table Information
Name              spark_catalog.mi_espacio.contribuyentes_lab02
Location
hdfs://namenode:8020/warehouse/iceberg/mi_espacio.db/contribuyentes_lab02
Provider          iceberg
Owner             root
Table Properties  [current-snapshot-
id=5030667150337317595,format=iceberg/parquet,format-version=1]
20 filas.
```

**Cómo se lee.** Primero lo simple. Las tres columnas son de texto, la tabla no tiene particiones, vive en una carpeta del almacenamiento y es Iceberg.

Ahora el detalle. La salida tiene tres columnas, `col_name`, `data_type` y `comment`, y se lee por bloques.

- Las tres primeras filas son tus columnas, `rut`, `razon_social` y `segmento`, todas `string`, que es texto.
- `# Partitioning` y `Not partitioned` dicen que la tabla no está repartida en cajones.
- `# Metadata Columns` lista columnas que Iceberg agrega por su cuenta y que se pueden consultar aunque no las creaste. `_spec_id` es el número del criterio de partición con que se escribió cada fila. `_partition` es el cajón de la fila, que aquí sale vacío, `struct<>`, porque no hay cajones. `_file` es el papelito donde está la fila. `_pos` es la posición de la fila dentro de ese papelito. `_deleted` dice si la fila está marcada como borrada.
- `# Detailed Table Information` abre el bloque de la portada.
- `Name` es el nombre completo, `spark_catalog.mi_espacio.contribuyentes_lab02`, con el catálogo, el espacio y la tabla.
- `Location` es la carpeta donde viven los archivos de la tabla. `hdfs://namenode:8020` es el almacenamiento del ambiente.
- `Provider` dice `iceberg`.
- `Owner` es el usuario dueño de la tabla, `root`, que es el usuario con que corre el cuaderno.
- `Table Properties` trae las propiedades, que la celda siguiente muestra ordenadas.

Las filas en blanco solo separan bloques. El cuaderno las cuenta, por eso dice `20 filas`.

## Celda 1.2 · Las propiedades

**La pregunta.** ¿Qué propiedades tiene la tabla?

**Por qué ahora.** Entre ellas está la que dice qué página manda.

**En el almacén.** Es leer en la portada qué página está marcada como la vigente.

**La sentencia, parte por parte.**

- `SHOW TBLPROPERTIES` lista las propiedades de la tabla, cada una como clave y valor.
- `contribuyentes_lab02` es la tabla.

```
sql
%%sql
-- Celda 1.2
SHOW TBLPROPERTIES contribuyentes_lab02
```

**Lo que sale en pantalla.**

```
text
key          value
current-snapshot-id 5030667150337317595
format       iceberg/parquet
format-version 1
3 filas.
```

**Cómo se lee.** Tres propiedades.

- `current-snapshot-id` es la página que manda ahora, `5030667150337317595`.
- `format` dice `iceberg/parquet`. La tabla es Iceberg y sus datos están en archivos Parquet.
- `format-version` es la versión del formato Iceberg, la 1.

## Paso 2 · Las páginas y los manifiestos

### Celda 2.1 · Las páginas

**La pregunta.** ¿Qué páginas tiene la libreta?

**Por qué ahora.** Cargaste dos veces, así que tiene que haber dos páginas.

**En el almacén.** Es mirar la lista de páginas de la libreta.

**La sentencia, parte por parte.**

- `SELECT snapshot_id, committed_at, operation` pide el número de la página, la hora en que se escribió, en UTC, y qué se hizo.
- `FROM mi_espacio.contribuyentes_lab02.snapshots` es la vista de sistema con la lista de páginas. Se nombra con tres partes, espacio, tabla y vista, aunque ya hayas hecho `USE`. Con solo dos partes, SQL creería que `contribuyentes_lab02` es un espacio.
- `ORDER BY committed_at` ordena de la más vieja a la más nueva.

```
sql
%%sql
-- Celda 2.1
SELECT snapshot_id, committed_at, operation
FROM mi_espacio.contribuyentes_lab02.snapshots
ORDER BY committed_at
```

**Lo que sale en pantalla.**

```
text
snapshot_id      committed_at      operation
262060709377412523  2026-09-25 18:06:09.756  append
5030667150337317595  2026-09-25 18:06:10.106  append
2 filas.
```

**Cómo se lee.** Dos páginas, las dos `append`, una por cada carga. La primera, `262060709377412523`, es de las 18:06:09.756 UTC, las 15:06 en Chile. La segunda, `5030667150337317595`, es de las 18:06:10.106 UTC, y es la misma que `current-snapshot-id` dijo que manda.

### Celda 2.2 · El historial

**La pregunta.** ¿Qué página mandó en cada momento?

**Por qué ahora.** Las páginas dicen qué se escribió. El historial dice desde cuándo mandó cada una.

**En el almacén.** Es mirar el registro de cuándo cada página pasó a ser la vigente.

**La sentencia, parte por parte.**

- `made_current_at` es cuándo la página pasó a mandar, en UTC.
- `snapshot_id` es el número de la página.
- `is_current_ancestor` dice si la página está en el camino que lleva a la que manda hoy.
- `FROM mi_espacio.contribuyentes_lab02.history` es la vista del historial, con sus tres partes.
- `ORDER BY made_current_at` ordena de lo más viejo a lo más nuevo.

```

sql
%%sql
-- Celda 2.2
SELECT made_current_at, snapshot_id, is_current_ancestor
FROM mi_espacio.contribuyentes_lab02.history
ORDER BY made_current_at

```

Lo que sale en pantalla.

```

text
made_current_at    snapshot_id    is_current_ancestor
2026-09-25 18:06:09.756 262060709377412523 True
2026-09-25 18:06:10.106 5030667150337317595 True
2 filas.

```

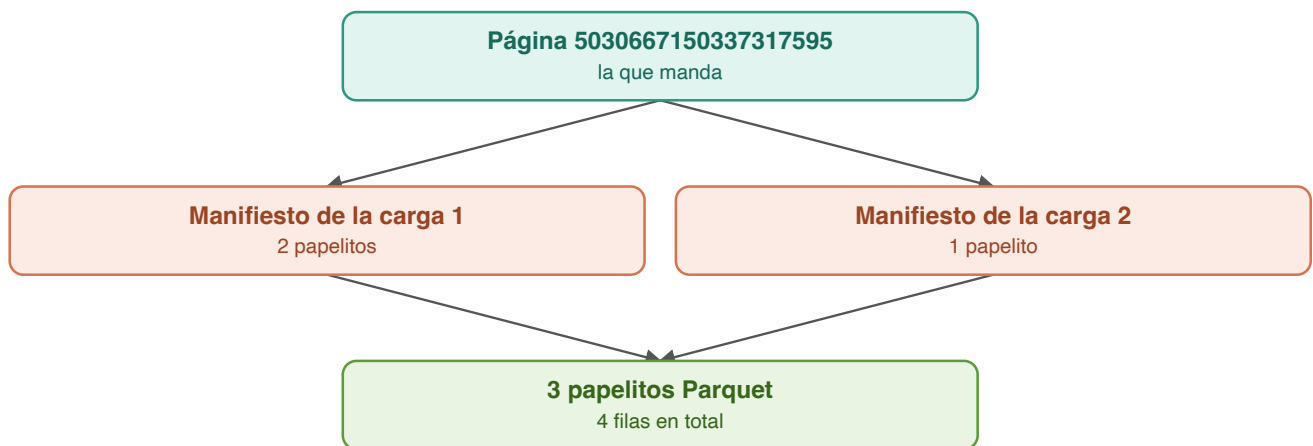
**Cómo se lee.** Las dos páginas, en el mismo orden y con las mismas horas, y las dos con `True`. La historia es una sola línea.

## Celda 2.3 · Los manifiestos

**La pregunta.** ¿Cómo sabe la tabla qué papelitos pertenecen a cada página?

**Por qué ahora.** Ya viste las páginas. Falta ver la lista de papelitos de cada una.

**En el almacén.** Es mirar las hojas que dicen, página por página, qué anotaciones le pertenecen. Con eso no hace falta revisar todo el estante para saber qué leer.



**Página 5030667150337317595** (verde agua). La página que manda nombra dos manifiestos, el de cada carga.

**Manifiesto de la carga 1** y **Manifiesto de la carga 2** (coral). Cada manifiesto es una lista de papelitos. El de la primera carga lista dos y el de la segunda, uno.

**3 papelitos Parquet** (verde). Son los archivos con las cuatro filas.

**La sentencia, parte por parte.**

- `path` es la ruta del archivo del manifiesto.
- `added_snapshot_id` es la página que lo agregó.
- `added_data_files_count` es cuántos papelitos agregó esa página.
- `existing_data_files_count` es cuántos papelitos de páginas anteriores repite el manifiesto.
- `FROM mi_espacio.contribuyentes_lab02.manifests` es la vista de los manifiestos de la página que manda.

```
sql
%%sql
-- Celda 2.3
SELECT path, added_snapshot_id, added_data_files_count, existing_data_files_count
FROM mi_espacio.contribuyentes_lab02.manifests
```

**Lo que sale en pantalla.**

```
text
path
added_snapshot_id    added_data_files_count  existing_data_files_count
hdfs://namenode:8020/warehouse/iceberg/mi_espacio.db/contribuyentes_lab02/metadata/a9ee5c5a-01a4-47a7-9f2e-e3487988f683-m0.avro  5030667150337317595  1          0
hdfs://namenode:8020/warehouse/iceberg/mi_espacio.db/contribuyentes_lab02/metadata/6c7b461e-8c90-44cf-825d-83b917293b73-m0.avro  262060709377412523  2          0
2 filas.
```

**Cómo se lee.** Dos manifiestos, uno por carga.

- El primero lo agregó la página `5030667150337317595`, la segunda carga, y trae 1 papelito.
- El segundo lo agregó la página `262060709377412523`, la primera carga, y trae 2 papelitos.
- `existing_data_files_count` es 0 en los dos, porque ninguno repite papelitos de otro.

Las rutas terminan en `-m0.avro`. Los manifiestos son archivos Avro que viven en la carpeta `metadata` de la tabla. El nombre largo es un identificador al azar.

Hay algo que puede sorprender. La primera carga eran tres filas y dejó dos papelitos. Spark escribió esa carga con dos tareas en paralelo, y cada tarea deja su propio papelito.

## Paso 3 · Dónde viven los datos

### Celda 3.1 · Los papelitos

**La pregunta.** ¿Cuántos archivos tiene la tabla y cuánto pesa cada uno?

**Por qué ahora.** Es la pregunta que se hace un administrador todos los días.

**En el almacén.** Es contar los papelitos del estante y pesarlos.

**La sentencia, parte por parte.**

- `file_path` es la ruta de cada papelito.
- `record_count` es cuántas filas tiene.
- `file_size_in_bytes` es cuánto pesa, en bytes.
- `FROM mi_espacio.contribuyentes_lab02.files` es la vista de los papelitos de la página que manda.

```
sql
%%sql
-- Celda 3.1
SELECT file_path, record_count, file_size_in_bytes
FROM mi_espacio.contribuyentes_lab02.files
```

**Lo que sale en pantalla.**

```
text
file_path
record_count  file_size_in_bytes
hdfs://namenode:8020/warehouse/iceberg/mi_espacio.db/contribuyentes_lab02/data/00000-2-06b860a7-
9186-4d9a-b528-1a7653fb3c10-00001.parquet  1          1109
hdfs://namenode:8020/warehouse/iceberg/mi_espacio.db/contribuyentes_lab02/data/00000-0-8b04d64f-
71d5-4b0a-af77-d5a0c94df0b8-00001.parquet  1          1102
hdfs://namenode:8020/warehouse/iceberg/mi_espacio.db/contribuyentes_lab02/data/00001-1-8b04d64f-
71d5-4b0a-af77-d5a0c94df0b8-00001.parquet  2          1091
3 filas.
```

**Cómo se lee.** Tres papelitos, con 1, 1 y 2 filas. Pesan 1109 bytes, 1102 bytes y 1091 bytes, poco más de un milésimo de MB cada uno.

Todos están en la carpeta `data` de la tabla. El nombre de cada uno lo arma Spark. `00000-2-` y `00000-0-`, `00001-1-` son el número de la tarea que lo escribió, y lo que sigue es un identificador al azar. Los dos que comparten identificador, `8b04d64f`, son de la misma escritura, la primera carga.

## Celda 3.2 · El resumen

**La pregunta.** ¿Cuánto es todo junto?

**Por qué ahora.** En producción se mira el resumen, no archivo por archivo.

**En el almacén.** Es anotar al final del día cuántos papelitos hay, cuántas anotaciones y cuánto pesan.

**La sentencia, parte por parte.**

- `count(*)` cuenta los papelitos. `AS archivos` le pone nombre.
- `sum(record_count)` suma las filas. `AS filas` le pone nombre.
- `sum(file_size_in_bytes)` suma los bytes. `AS bytes` le pone nombre.
- `FROM mi_espacio.contribuyentes_lab02.files` es la misma vista.

```
sql
%%sql
-- Celda 3.2
SELECT count(*)           AS archivos,
       sum(record_count)   AS filas,
       sum(file_size_in_bytes) AS bytes
FROM mi_espacio.contribuyentes_lab02.files
```

**Lo que sale en pantalla.**

```
text
archivos  filas  bytes
3         4     3302
1 fila.
```

**Cómo se lee.** Tres archivos, cuatro filas y 3302 bytes, que son 0,0033 MB.

La tabla es eso, tres papelitos Parquet más la portada y los manifiestos que dicen cuáles cuentan. No hay un servidor guardando el estado ni una base de datos escondida.

## Paso 4 · El mismo dato desde otro motor

Aquí trabajas en dos lugares. Primero en Hue, en el navegador, y después vuelves al cuaderno.

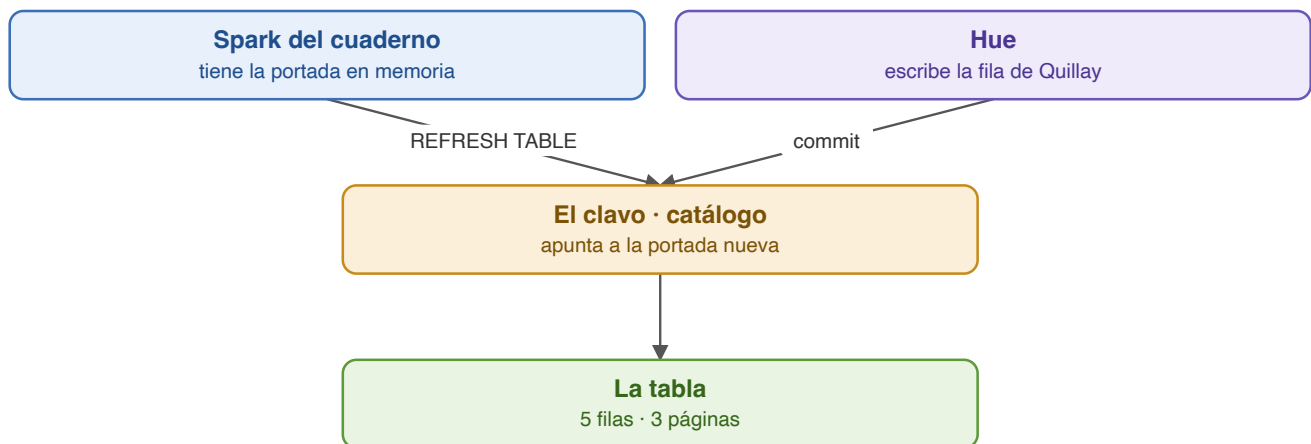
**En Hue.** Abre `http://localhost:8889/` en otra pestaña del navegador, entra con tu usuario y abre el editor **SparkSQL (Iceberg)**. Hue solo está arriba si levantaste el ambiente con `bin/ambiente.sh` arriba `--sql`. En Hue el nombre de la tabla va completo, con tu espacio adelante.

Primero consulta la tabla. Vas a ver las mismas cuatro filas que en el cuaderno.

```
sql
SELECT * FROM mi_espacio.contribuyentes_lab02 ORDER BY rut
```

Después agrega una fila, la de Quillay Servicios. Hue responde con una tabla vacía, sin filas, porque un `INSERT` no devuelve datos.

```
sql
INSERT INTO mi_espacio.contribuyentes_lab02 VALUES ('79412337-8','Quillay Servicios
SpA','PEQUENA')
```



**Spark del cuaderno** (azul). Guardó en memoria la portada que leyó la última vez. Hasta que refresque, sigue mirando esa.

**Hue** (morado). Escribe la fila y cuelga una página nueva, con su propia portada.

**El clavo · catálogo** (amarillo). Después del commit de Hue, apunta a la portada nueva.

**La tabla** (verde). Queda con cinco filas y tres páginas, sin que nadie copie nada entre los motores.

### Celda 4.1 · Mirar desde el cuaderno

**La pregunta.** ¿El cuaderno ya ve la fila que escribiste en Hue?

**Por qué ahora.** Hue ya hizo su commit. Es el momento de mirar desde el otro motor.

**En el almacén.** Es volver a tu mostrador y leer la libreta que tenías abierta sobre la mesa.

**La sentencia, parte por parte.**

- `SELECT * FROM contribuyentes_lab02 ORDER BY rut` lee la tabla completa, ordenada por RUT.

```
sql
%%sql
-- Celda 4.1
SELECT * FROM contribuyentes_lab02 ORDER BY rut
```

**Lo que sale en pantalla.**

```
text
rut          razon_social          segmento
76171162-8   Huemul Alimentos EIRL            GRANDE
77746521-K   Pehuen Logistica EIRL            MICRO
77884562-8   Araucaria Ferreteria SpA         PEQUENA
78800840-6   Copihue Maquinarias EIRL         MEDIANA
4 filas.
```

**Cómo se lee.** Todavía cuatro filas, no cinco. El dato ya está escrito, pero este motor muestra lo que tenía en memoria.

Para no ir al catálogo en cada consulta, Spark se guarda una copia de la portada que leyó. Esa copia se vence sola al rato de no usarse, pero no hace falta esperarla, se le pide que refresque. Esto no es coordinarse con el otro motor. El dato quedó escrito desde el momento en que Hue hizo su commit, y el único atrasado era el cuaderno.

## Celda 4.2 · Refrescar

**La pregunta.** ¿Cómo le pides al motor que vuelva a leer la portada?

**Por qué ahora.** La copia en memoria está vieja.

**En el almacén.** Es dejar la libreta que tenías en la mesa y volver a mirar la que cuelga del clavo.

**La sentencia, parte por parte.**

- `REFRESH TABLE` bota la copia en memoria de la tabla. La próxima consulta vuelve a preguntar al catálogo cuál es la portada vigente.
- `contribuyentes_lab02` es la tabla.

```
sql
%%sql
-- Celda 4.2
REFRESH TABLE contribuyentes_lab02
```

**Lo que sale en pantalla.**

```
text
Listo. La sentencia se ejecutó.
```

**Cómo se lee.** Terminó bien. No devuelve datos, solo borra la copia.

## Celda 4.3 · Mirar de nuevo

**La pregunta.** ¿Ahora sí aparece la fila de Hue?

**Por qué ahora.** Ya se refrescó la copia.

**En el almacén.** Es leer la libreta que cuelga del clavo, con lo que anotó el otro.

**La sentencia, parte por parte.**

- `SELECT * FROM contribuyentes_lab02 ORDER BY rut` es la misma consulta de la celda 4.1.

```
sql
%%sql
-- Celda 4.3
SELECT * FROM contribuyentes_lab02 ORDER BY rut
```

**Lo que sale en pantalla.**

```
text
rut          razon_social          segmento
76171162-8   Huemul Alimentos EIRL            GRANDE
77746521-K   Pehuen Logistica EIRL            MICRO
77884562-8   Araucaria Ferreteria SpA         PEQUENA
78800840-6   Copihue Maquinarias EIRL         MEDIANA
79412337-8   Quillay Servicios SpA           PEQUENA
5 filas.
```

**Cómo se lee.** Cinco filas. La quinta es Quillay Servicios, RUT `79412337-8`, la que escribiste en Hue. No exportaste, no copiaste y no sincronizaste nada entre los dos motores.

## Celda 4.4 · Las páginas, otra vez

**La pregunta.** ¿Qué dejó en la libreta la escritura de Hue?

**Por qué ahora.** Para ver que Spark no fue a preguntarle a Hue. Leyó la tabla.

**En el almacén.** Es mirar de nuevo la lista de páginas.

**La sentencia, parte por parte.**

- `SELECT snapshot_id, committed_at, operation` pide número, hora y operación de cada página.
- `FROM mi_espacio.contribuyentes_lab02.snapshots` es la vista de las páginas.
- `ORDER BY committed_at` ordena de la más vieja a la más nueva.

```
sql
%%sql
-- Celda 4.4
SELECT snapshot_id, committed_at, operation
FROM mi_espacio.contribuyentes_lab02.snapshots
ORDER BY committed_at
```

## Lo que sale en pantalla.

```
text
snapshot_id      committed_at      operation
262060709377412523  2026-09-25 18:06:09.756  append
5030667150337317595  2026-09-25 18:06:10.106  append
3392707437724357021  2026-09-25 18:06:14.184  append
3 filas.
```

**Cómo se lee.** Tres páginas. Las dos primeras son tus cargas. La tercera, `3392707437724357021`, de las 18:06:14.184 UTC, es la que escribió Hue. Para la tabla, esa escritura es una página más, igual que las tuyas.

# Preguntas frecuentes

## ¿Por qué hay que nombrar las vistas con tres partes aunque ya hice `USE` ?

Porque `contribuyentes_lab02.snapshots` tiene dos partes, y SQL lee las dos partes como espacio y tabla. Buscaría una tabla `snapshots` en un espacio `contribuyentes_lab02`, que no existe. Con tres partes, `mi_espacio.contribuyentes_lab02.snapshots`, queda claro que la tercera es la vista.

## ¿Dónde están físicamente la portada y los manifiestos?

En la carpeta de la tabla, dentro de `metadata`. Los papelitos de datos están en `data`. Los dos están en la ruta que muestra `Location` en la celda 1.1.

## ¿Por qué tres papelitos para cuatro filas?

Porque cada escritura deja al menos un papelito, y Spark puede repartir una escritura entre varias tareas. La primera carga se escribió con dos tareas y dejó dos. Con pocas filas no importa. Con muchas escrituras chicas, la tabla se llena de papelitos, y eso sí importa.

## ¿Por qué el cuaderno mostró cuatro filas si Hue ya había escrito la quinta?

Porque Spark tenía en memoria la portada que había leído antes. `REFRESH TABLE` la bota y la próxima consulta lee la vigente. Si esperas un rato sin consultar la tabla, la copia también se vence sola.

## ¿Hue y el cuaderno se hablan?

No. Hue escribe a través del Spark Thrift Server, que es otro proceso de Spark, y el cuaderno tiene el suyo. Los dos miran el mismo catálogo y la misma carpeta. Por eso ven lo mismo sin coordinarse.

## ¿Qué pasa si escribo en Hue otra fila distinta?

La tabla queda con esa fila en vez de la de Quillay, y la celda 4.3 muestra lo que escribiste. Las páginas son las mismas tres.

## Cierre

**La pregunta.** ¿Qué te llevas de este laboratorio?

Una tabla Iceberg es un conjunto de archivos guardados, ninguno dentro de un motor. Por eso dos motores que miran el mismo catálogo leen y escriben la misma tabla.

**En el almacén.** No hay dos libretas. Hay una sola, colgada del mismo clavo, y cualquiera que mire el clavo lee la misma.

| En el almacén         | En Iceberg       | Qué contiene                        |
|-----------------------|------------------|-------------------------------------|
| El clavo en la pared  | Catálogo         | cuál portada es la vigente          |
| La portada            | metadata.json    | columnas, páginas, la que manda     |
| La página fechada     | Snapshot         | un estado de la tabla               |
| La lista de papelitos | Manifiesto       | qué archivos pertenecen a la página |
| Los papelitos         | Archivos Parquet | los datos                           |

Los números de la solución ejecutada.

| Momento                    | Filas | Páginas |
|----------------------------|-------|---------|
| Después de las dos cargas  | 4     | 2       |
| Después de escribir en Hue | 5     | 3       |

Spark y Hue no se hablan y no lo necesitan. Los dos miran el mismo clavo y leen la misma libreta.